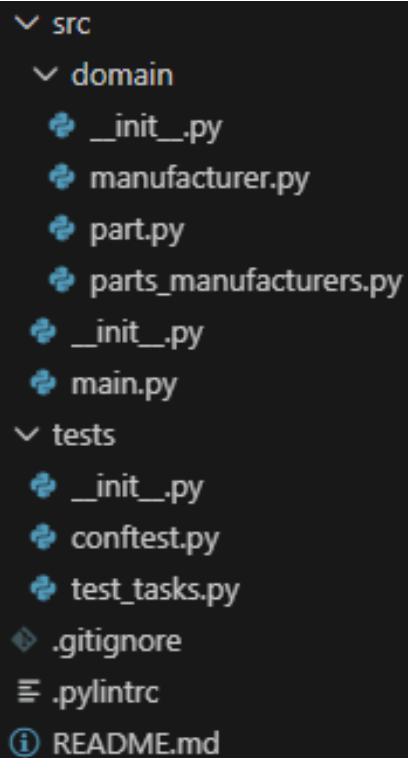


Рубежный контроль №2 по дисциплине ПиКЯП

Никандров Валентин ИУ5-35Б

Вариант №19 («Г»)

Файловая структура проекта:



```

└─ src
   └─ domain
      ├── __init__.py
      ├── manufacturer.py
      ├── part.py
      ├── parts_manufacturers.py
      ├── __init__.py
      └── main.py
   └─ tests
      ├── __init__.py
      ├── conftest.py
      └── test_tasks.py
├─ .gitignore
├─ .pylintrc
└─ README.md

```

Листинг кода проекта:

```
"""Main application module"""

from src.domain import Part, Manufacturer, PartsManufacturers

parts = [
    Part(0, 0, "Поршень двигателя V8", 450, "Двигатель", 2500),
    Part(1, 1, "Турбокомпрессор ТКР-7", 3200, "Система наддува", 18500),
    Part(2, 1, "Катализатор", 2800, "Выхлопная система", 12200),
    Part(3, 2, "Рычаг подвески верхний", 1850, "Подвеска", 4700),
    Part(4, 3, "Фара светодиодная би-ксенон", 950, "Освещение", 8900),
]

manufacturers = [
    Manufacturer(
        0,
        "Bosch Автокомпоненты",
        "Автомобильные компоненты",
        "Системы впрыска",
        500000,
        "+7 (495) 123-45-67",
        "bosch@example.com",
    ),
    Manufacturer(
        1,
        "Garrett Motion",
        "Автомобильные системы",
        "Турбонаддув",
        200000,
        "+7 (495) 234-56-78",
        "garrett@example.com",
    ),
    Manufacturer(
        2,
        "ZF Friedrichshafen",
        "Автомобильные системы",
        "Подвеска и трансмиссия",
        300000,
        "+7 (495) 456-78-90",
        "zf@example.com",
    ),
    Manufacturer(
        3,
        "HELLA Russia",
        "Автомобильная электроника",
        "Освещение",
        350000,
        "+7 (495) 567-89-01",
        "hella@example.com",
    ),
]
```

```

    ),
]

PartsManufacturers = [
    PartsManufacturers(0, 0),
    PartsManufacturers(1, 1),
    PartsManufacturers(2, 1),
    PartsManufacturers(3, 2),
    PartsManufacturers(4, 3),
    PartsManufacturers(0, 2),
]

def first_task(original_one_to_many):
    """Return manufacturers starting with 'G' and their parts."""
    print("\nЗадание 1:")

    one_to_many = sorted(original_one_to_many, key=lambda x:
x["manufacturer_id"])

    manufacturer_parts = []

    for part in one_to_many:
        manufacturer_name = part["manufacturer_name"]
        if manufacturer_name[0] == "G":
            manufacturer_parts.append(part)

    return manufacturer_parts

def print_first_task_result(manufacturer_parts):
    """Print manufacturers starting with 'G' and their parts."""
    previous_manufacturer_name = ""

    for part in manufacturer_parts:
        manufacturer_name = part["manufacturer_name"]
        if not manufacturer_name or manufacturer_name[0] != "G":
            continue
        if manufacturer_name != previous_manufacturer_name:
            previous_manufacturer_name = manufacturer_name
            print(f"\nПроизводитель {manufacturer_name}:")

        part_name = part["name"]
        print(part_name)

def second_task(original_one_to_many):
    """Return the most expensive part for each manufacturer."""

    one_to_many = sorted(
        original_one_to_many, key=lambda x: x["production_cost"], reverse=True

```

```

)

most_expensive_parts = []
previous_manufacturers_ids = set()

for part in one_to_many:
    manufacturer_id = part["manufacturer_id"]
    if manufacturer_id not in previous_manufacturers_ids:
        previous_manufacturers_ids.add(manufacturer_id)

        manufacturer_name = part["manufacturer_name"]
        name = part["name"]
        production_cost = part["production_cost"]

        most_expensive_parts.append(
            {
                "manufacturer_name": manufacturer_name,
                "name": name,
                "production_cost": production_cost,
            }
        )

return most_expensive_parts

def print_second_task_result(most_expensive_parts):
    """Print the most expensive part for each manufacturer."""
    print("\nЗадание Г2:")

    for part in most_expensive_parts:
        print(
            f"Производитель: {part['manufacturer_name']}; "
            f"Запчасть: {part['name']}; "
            f"Себестоимость производства: {part['production_cost']}"
        )

def third_task(original_many_to_many):
    """Return manufacturers and their parts sorted by production capacity."""
    many_to_many = sorted(
        original_many_to_many,
        key=lambda x: x["manufacturer_production_capacity"],
        reverse=True,
    )

    manufacturers = dict()

    for manufacturer_part in many_to_many:
        if manufacturer_part["manufacturer_id"] not in manufacturers:
            manufacturers[manufacturer_part["manufacturer_id"]] = {
                "manufacturer_name": manufacturer_part["manufacturer_name"],

```

```

        "manufacturer_production_capacity": manufacturer_part[
            "manufacturer_production_capacity"
        ],
        "parts": [],
    }

    manufacturers[manufacturer_part["manufacturer_id"]]["parts"].append(
        manufacturer_part["part_name"]
    )

    return manufacturers

def print_third_task_result(manufacturers):
    print("\nЗадание Г3:")

    previous_manufacturer_name = ""

    for manufacturer in manufacturers.values():
        manufacturer_name = manufacturer["manufacturer_name"]

        if manufacturer_name != previous_manufacturer_name:
            previous_manufacturer_name = manufacturer["manufacturer_name"]
            manufacturer_production_capacity = manufacturer[
                "manufacturer_production_capacity"
            ]

            print(
                f"\nПроизводитель {manufacturer_name} (Производственная "
                f"мощность {manufacturer_production_capacity}):"
            )

            for part in manufacturer["parts"]:
                print(part)

def main():
    """Main function."""
    one_to_many = [
        {
            "name": part.name,
            "production_cost": part.production_cost,
            "manufacturer_name": manufacturer.name,
            "manufacturer_id": manufacturer.id,
        }
        for manufacturer in manufacturers
        for part in parts
        if part.manufacturer_id == manufacturer.id
    ]

    print_first_task_result(first_task(one_to_many))

```

```

print_second_task_result(second_task(one_to_many))

many_to_many_temp = [
    {
        "manufacturer_id": part_manufacturer.manufacturer_id,
        "part_id": part_manufacturer.part_id,
        "manufacturer_name": manufacturer.name,
        "manufacturer_production_capacity": manufacturer.production_capacity,
    }
    for manufacturer in manufacturers
    for part_manufacturer in PartsManufacturers
    if manufacturer.id == part_manufacturer.manufacturer_id
]

many_to_many = [
    {
        "manufacturer_id": manufacturer_part["manufacturer_id"],
        "manufacturer_name": manufacturer_part["manufacturer_name"],
        "manufacturer_production_capacity": manufacturer_part[
            "manufacturer_production_capacity"
        ],
        "part_name": part.name,
    }
    for manufacturer_part in many_to_many_temp
    for part in parts
    if part.id == manufacturer_part["part_id"]
]

print_third_task_result(third_task(many_to_many))

if __name__ == "__main__":
    main()

```

```

"""Domain entities module for task implementation algorithm.

This is module provides domain subject enteties.

Classes:
    Part: Represents a manufacturing part with attributes.
    Manufacturer: Represents a manufacturer with attributes.
    PartsManufacturers: Represents a relationships between parts and
manufacturers
"""

from .part import Part
from .manufacturer import Manufacturer
from .parts_manufacturers import PartsManufacturers

```

```
__all__ = ["Part", "Manufacturer", "PartsManufacturers"]
__version__ = "0.1.0"

def get_version():
    return __version__
```

```
"""Module for representing manufacturing parts."""

from dataclasses import dataclass, field

@dataclass(order=True)
class Part:
    """Represents a manufacturing part.

    Attributes:
        id: Unique identifier for the part.
        manufacturer_id: identifier for the manufacturer.
        name: Name of the part.
        weight: Weight of the part in grams.
        category: Category of the part.
        production_cost: Production cost in currency units.
    """

    id: int = field(compare=False)
    manufacturer_id: int = field(compare=False)
    name: str = field(compare=False)
    weight: int = field(compare=False)
    category: str = field(compare=False)
    production_cost: int = field(compare=True)

    def __eq__(self, other):
        """Checks equality based on part attributes."""
        if not isinstance(other, type(self)):
            return NotImplemented
        return (
            self.manufacturer_id == other.manufactorer_id
            and self.name == other.name
            and self.weight == other.weight
            and self.category == other.category
            and self.production_cost == other.production_cost
        )
```

```
"""Module for representing manufacturers."""

from dataclasses import dataclass, field
```

```

@dataclass(order=True)
class Manufacturer:
    """Represents a manufacturer.

    Attributes:
        id (int): Unique identifier for the manufacturer.
        name (str): Name of the manufacturer.
        industry (str): Industry sector of the manufacturer.
        specialization (str): Specialization area of the manufacturer.
        production_capacity (int): Production capacity in units per time period.
        phone (str): Contact phone number.
        email (str): Contact email address.
    """

    id: int = field(compare=False)
    name: str = field(compare=False)
    industry: str = field(compare=False)
    specialization: str = field(compare=False)
    production_capacity: int = field(compare=True)
    phone: str = field(compare=False)
    email: str = field(compare=False)

    def __eq__(self, other):
        """Checks equality based on manufacturer attributes."""
        if not isinstance(other, type(self)):
            return NotImplemented
        return (
            self.name == other.name
            and self.industry == other.industry
            and self.specialization == other.specialization
            and self.production_capacity == other.production_capacity
            and self.email == other.email
            and self.phone == other.phone
        )

```

```

"""Module for representing relationships."""

from dataclasses import dataclass

@dataclass
class PartsManufacturers:
    """Represents a many-to-many relationship between parts and manufacturers.

    Attributes:
        part_id (int): The unique identifier of the part.
        manufacturer_id (int): The unique identifier of the manufacturer.
    """

```



```
"""

part_id: int
manufacturer_id: int
```

```
"""
Pytest fixtures for testing the main application module.

Provides sample data for one-to-many and many-to-many relationships
between parts and manufacturers, to be used in unit tests.
"""

import pytest
from src.domain import Part, Manufacturer, PartsManufacturers

@pytest.fixture
def parts():
    return [
        Part(0, 0, "Гидравлический усилитель руля", 2000, "Рулевое управление",
9600),
        Part(1, 0, "Полуось привода", 3500, "Привод", 14300),
        Part(2, 1, "Тормозной суппорт передний", 1800, "Тормоза", 7800),
        Part(3, 2, "Радиатор охлаждения", 1200, "Охлаждение", 5400),
        Part(4, 2, "Датчик массового расхода воздуха", 400, "Датчики", 6200),
        Part(5, 3, "Амортизатор задний", 2500, "Подвеска", 8800),
    ]

@pytest.fixture
def manufacturers():
    return [
        Manufacturer(
            0,
            "GKN Automotive",
            "Автомобильные системы",
            "Рулевое управление",
            420000,
            "+7 111 111-11-11",
            "gkn@example.com",
        ),
        Manufacturer(
            1,
            "Girlock Systems",
            "Автомобильные системы",
            "Тормоза",
            180000,
            "+7 222 222-22-22",
            "girlock@example.com",
        ),
    ]
```

```

    ),
    Manufacturer(
        2,
        "Denso Corporation",
        "Автомобильные компоненты",
        "Охлаждение и датчики",
        510000,
        "+7 333 333-33-33",
        "denso@example.com",
    ),
    Manufacturer(
        3,
        "KYB Europe",
        "Автомобильные системы",
        "Подвеска",
        290000,
        "+7 444 444-44-44",
        "kyb@example.com",
    ),
]

```

```

@pytest.fixture
def parts_manufacturers():
    return [
        PartsManufacturers(0, 0),
        PartsManufacturers(1, 0),
        PartsManufacturers(2, 1),
        PartsManufacturers(3, 2),
        PartsManufacturers(4, 2),
        PartsManufacturers(5, 3),
    ]

```

```

@pytest.fixture
def one_to_many(parts, manufacturers):
    return [
        {
            "name": part.name,
            "production_cost": part.production_cost,
            "manufacturer_name": manufacturer.name,
            "manufacturer_id": manufacturer.id,
        }
        for manufacturer in manufacturers
        for part in parts
        if part.manufacturer_id == manufacturer.id
    ]

```

```

@pytest.fixture
def many_to_many(parts, manufacturers, parts_manufacturers):

```

```

many_to_many_temp = [
    {
        "manufacturer_id": part_manufacturer.manufacturer_id,
        "part_id": part_manufacturer.part_id,
        "manufacturer_name": manufacturer.name,
        "manufacturer_production_capacity": manufacturer.production_capacity,
    }
    for manufacturer in manufacturers
    for part_manufacturer in parts_manufacturers
    if manufacturer.id == part_manufacturer.manufacturer_id
]

return [
    {
        "manufacturer_id": manufacturer_part["manufacturer_id"],
        "manufacturer_name": manufacturer_part["manufacturer_name"],
        "manufacturer_production_capacity": manufacturer_part[
            "manufacturer_production_capacity"
        ],
        "part_name": part.name,
    }
    for manufacturer_part in many_to_many_temp
    for part in parts
    if part.id == manufacturer_part["part_id"]
]

```

"""Unit tests for the main application module.

This module provides tests for `first_task`, `second_task`, and `third_task` functions, using `pytest` fixtures for sample data of parts and manufacturers.

```

from src.main import first_task, second_task, third_task

```

```

def test_first_task_only(one_to_many):
    result = first_task(one_to_many)

    assert len(result) == 3
    assert all(item["manufacturer_name"].startswith("G") for item in result)

```

```

def test_second_task(one_to_many):
    result = second_task(one_to_many)

    manufacturers = {
        item["manufacturer_name"]: item["production_cost"] for item in result
    }

```

```
assert manufacturers["GKN Automotive"] == 14300
assert manufacturers["Denso Corporation"] == 6200
assert manufacturers["KYB Europe"] == 8800
assert manufacturers["Girlock Systems"] == 7800

def test_third_task_sorting_and_grouping(many_to_many):
    result = third_task(many_to_many)

    manufacturers = list(result.values())

    assert manufacturers[0]["manufacturer_name"] == "Denso Corporation"
    assert manufacturers[0]["manufacturer_production_capacity"] == 510000

    assert "Полуось привода" in manufacturers[1]["parts"]
```

Результат работы программы:

```
vIntester@DESKTOP-ACISLUE:~/Nikandrov-PCPL-Labs-2025$ python3 -m src.main
```

Задание Г1:

Производитель Garrett Motion:
Турбокомпрессор ТКР-7
Катализатор

Задание Г2:

Производитель: Garrett Motion; Запчасть: Турбокомпрессор ТКР-7; Себестоимость производства: 18500
Производитель: HELLA Russia; Запчасть: Фара светодиодная би-ксенон; Себестоимость производства: 8900
Производитель: ZF Friedrichshafen; Запчасть: Рычаг подвески верхний; Себестоимость производства: 4700
Производитель: Bosch Автокомпоненты; Запчасть: Поршень двигателя V8; Себестоимость производства: 2500

Задание Г3:

Производитель Bosch Автокомпоненты (Производственная мощность 500000):
Поршень двигателя V8

Производитель HELLA Russia (Производственная мощность 350000):
Фара светодиодная би-ксенон

Производитель ZF Friedrichshafen (Производственная мощность 300000):
Рычаг подвески верхний
Поршень двигателя V8

Производитель Garrett Motion (Производственная мощность 200000):
Турбокомпрессор ТКР-7
Катализатор

```
vIntester@DESKTOP-ACISLUE:~/Nikandrov-PCPL-Labs-2025$ pytest
```

```
===== test session starts =====
```

```
platform linux -- Python 3.12.3, pytest-7.4.4, pluggy-1.4.0  
rootdir: /home/vIntester/Nikandrov-PCPL-Labs-2025  
collected 3 items
```

```
tests/test_tasks.py ...
```

```
[100%]
```

```
===== 3 passed in 0.02s =====
```